Docker & Kubernetes for Developers

Session 06: Konfigürasyon Yönetimi ve Sırlar

Eğitmen: Hüseyin Babal
Süre: 2 Saat (Teori + Hands-on Lab)

📋 Bu Oturum İçeriği

- ✅ ConfigMaps ve configuration management
- ✅ Secrets ve sensitive data güvenliği
- ✅ Environment variables ve configuration injection
- ✅ Configuration patterns ve best practices
- ✅ Security best practices
- ✅ **E-commerce microservices configuration (Lab)**

⚙️ Configuration Management Challenges

Traditional Challenges:

- **Hardcoded values** in application code
- **Environment-specific** configurations
- **Secret management** complexity
- **Configuration drift** between environments
- **Hot reloading** configuration changes

Kubernetes Solution:

```
ConfigMaps (Non-sensitive) + Secrets (Sensitive) = ✅
```

Separation of Concerns: Code ↔️ Configuration

📄 ConfigMaps Nedir?

> **ConfigMap**, key-value pair'lar halinde configuration data'sını tutar ve Pod'lara inject eder.

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  database.host: "mysql-service"
  database.port: "3306"
  log.level: "INFO"
  features.enabled: "true"
  app.properties: |
    server.port=8080
    spring.datasource.url=jdbc:mysql://mysql:3306/appdb
    logging.level.com.app=DEBUG
```

ConfigMap Kullanım Yolları:

1. **Environment Variables**
2. **Volume Mounts** (files)
3. **Command Line Arguments**

🌍 ConfigMap - Environment Variables

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
spec:
  template:
    spec:
      containers:
      - name: user-service
        image: user-service:latest
        env:
        - name: DATABASE_HOST
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: database.host
        - name: DATABASE_PORT
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: database.port
        envFrom:  # Tüm ConfigMap'i environment variable olarak inject et
        - configMapRef:
            name: app-config
```

📁 ConfigMap - Volume Mount

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
spec:
  template:
    spec:
      containers:
      - name: user-service
        image: user-service:latest
        volumeMounts:
        - name: config-volume
          mountPath: /etc/config
          readOnly: true
      volumes:
      - name: config-volume
        configMap:
          name: app-config
          items:
          - key: app.properties
            path: application.properties
```

Result: `/etc/config/application.properties` dosyası oluşur

🔐 Secrets - Sensitive Data

> **Secret**, şifreler, token'lar, key'ler gibi hassas bilgileri base64 encoded olarak tutar.

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: dXNlcg==      # base64("user")
  password: cGFzc3dvcmQ=  # base64("password")
stringData:  # Plain text (automatically encoded)
  api-key: "my-secret-api-key"
  jwt-secret: "super-secret-jwt-signing-key"
```

Secret Types:

- **Opaque:** Generic secret
- **kubernetes.io/service-account-token**
- **kubernetes.io/dockercfg:** Docker registry
- **kubernetes.io/tls:** TLS certificates

🔑 Secret Usage Patterns

Environment Variables

```yaml
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: username
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: password
```

Volume Mounts

```yaml
volumeMounts:
- name: secret-volume
  mountPath: /etc/secrets
  readOnly: true
volumes:
- name: secret-volume
  secret:
    secretName: db-secret
```

🎯 Configuration Injection Patterns

Pattern 1: Environment Variables

- ✅ **Pros:** Simple, widely supported
- ❌ **Cons:** Visible in process list, limited size

Pattern 2: Volume Mounts

- ✅ **Pros:** File-based config, supports hot reload
- ❌ **Cons:** Requires application file watching

Pattern 3: Init Containers

- ✅ **Pros:** Configuration preprocessing
- ❌ **Cons:** Complex setup

Pattern 4: Sidecar Configuration

- ✅ **Pros:** Dynamic configuration updates
- ❌ **Cons:** Resource overhead

🧪 Lab: E-commerce Configuration

User Service ConfigMap

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: user-service-config
data:
  application.yml: |
    server:
      port: 8080
    spring:
      application:
        name: user-service
      datasource:
        url: jdbc:mysql://mysql:3306/userdb
        driver-class-name: com.mysql.cj.jdbc.Driver
      jpa:
        hibernate:
          ddl-auto: update
        show-sql: false
      rabbitmq:
        host: rabbitmq
        port: 5672
        virtual-host: ecommerce
    logging:
      level:
        com.ecommerce.userservice: INFO
```

🔐 Lab: Database Secrets

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: database-secrets
type: Opaque
stringData:
  # MySQL credentials
  mysql-root-password: "rootpassword"
  mysql-user: "userservice"
  mysql-password: "password"
  mysql-database: "userdb"

  # PostgreSQL credentials
  postgres-user: "productservice"
  postgres-password: "password"
  postgres-database: "productdb"

  # MongoDB credentials
  mongo-root-user: "root"
  mongo-root-password: "rootpassword"
  mongo-database: "orderdb"
```

🔑 Lab: Application Secrets

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
type: Opaque
stringData:
  # JWT secrets
  jwt-secret: "my-super-secret-jwt-signing-key-2024"
  jwt-expiration: "86400000"

  # RabbitMQ credentials
  rabbitmq-user: "admin"
  rabbitmq-password: "password"
  rabbitmq-vhost: "ecommerce"

  # Redis credentials
  redis-password: "password"

  # External API keys
  stripe-api-key: "sk_test_your_stripe_api_key"
  sendgrid-api-key: "SG.your_sendgrid_api_key"
```

🚀 Lab: User Service with Configuration

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
      - name: user-service
        image: user-service:v1.0.0
        ports:
        - containerPort: 8080
        env:
        - name: DB_HOST
          value: "mysql"
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: database-secrets
              key: mysql-user
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: database-secrets
              key: mysql-password
        - name: JWT_SECRET
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: jwt-secret
        volumeMounts:
        - name: config-volume
          mountPath: /app/config
          readOnly: true
      volumes:
      - name: config-volume
        configMap:
          name: user-service-config
```

Lab: Product Service Configuration

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: product-service-config
data:
  appsettings.json: |
    {
      "Logging": {
        "LogLevel": {
          "Default": "Information"
        }
      },
      "Database": {
        "Host": "postgresql",
        "Port": "5432",
        "Name": "productdb"
      },
      "RabbitMQ": {
        "Host": "rabbitmq",
        "Port": 5672,
        "VirtualHost": "ecommerce"
      }
    }

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
      - name: product-service
        image: product-service:v1.0.0
        ports:
        - containerPort: 8080
        env:
        - name: Database__User
          valueFrom:
            secretKeyRef:
              name: database-secrets
              key: postgres-user
        - name: Database__Password
          valueFrom:
            secretKeyRef:
              name: database-secrets
              key: postgres-password
        volumeMounts:
        - name: config-volume
          mountPath: /app/config
          readOnly: true
      volumes:
      - name: config-volume
        configMap:
          name: product-service-config
```

📦 Lab: Order Service Configuration

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: order-service-config
data:
  config.env: |
    PORT=8080
    MONGO_HOST=mongodb
    MONGO_PORT=27017
    MONGO_DB_NAME=orderdb
    RABBITMQ_HOST=rabbitmq
    RABBITMQ_PORT=5672

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
      - name: order-service
        image: order-service:v1.0.0
        ports:
        - containerPort: 8080
        envFrom:
        - configMapRef:
            name: order-service-config
        env:
        - name: MONGO_USERNAME
          valueFrom:
            secretKeyRef:
              name: database-secrets
              key: mongo-root-user
        - name: MONGO_PASSWORD
          valueFrom:
            secretKeyRef:
              name: database-secrets
              key: mongo-root-password
```

🔒 Security Best Practices

Secret Management:

- ❌ **Hardcode** secrets in Docker images
- ✅ Use **Kubernetes Secrets**
- ✅ Enable **encryption at rest** (etcd)
- ✅ Use **external secret management** (Vault, AWS Secrets Manager)
- ✅ **Rotate secrets** regularly

ConfigMap Best Practices:

- ✅ **Version** your configurations
- ✅ Use **separate ConfigMaps** per service
- ✅ **Validate** configuration before deployment
- ✅ **Monitor** configuration changes

🔐 Advanced Secret Management

External Secrets Operator

```yaml
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: vault-backend
spec:
  provider:
    vault:
      server: "https://vault.example.com"
      path: "secret"
      auth:
        kubernetes:
          mountPath: "kubernetes"
          role: "example-role"

---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: vault-secret
spec:
  secretStoreRef:
    name: vault-backend
    kind: SecretStore
  target:
    name: app-secret
  data:
  - secretKey: password
    remoteRef:
      key: database
```

🔄 Configuration Hot Reload

ConfigMap Update Detection

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: config-reload-example
spec:
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - name: config
      mountPath: /etc/nginx/conf.d
      readOnly: true
  - name: config-reloader  # Sidecar
    image: configmap-reload:latest
    args:
    - --volume-dir=/etc/nginx/conf.d
    - --webhook-url=http://localhost:80/-/reload
    volumeMounts:
    - name: config
      mountPath: /etc/nginx/conf.d
      readOnly: true
  volumes:
  - name: config
    configMap:
      name: nginx-config
```

🧪 Lab: Configuration Testing

```
# ConfigMap ve Secret oluşturma
kubectl apply -f configmaps/
kubectl apply -f secrets/

# Configuration'ları kontrol etme
kubectl get configmaps
kubectl describe configmap user-service-config

kubectl get secrets
kubectl describe secret database-secrets

# Pod içinde configuration test
kubectl exec -it user-service-xxxx -- env | grep DB
kubectl exec -it user-service-xxxx -- cat /app/config/application.yml

# Configuration update test
kubectl edit configmap user-service-config
# Wait ~60 seconds for kubelet to update volume
kubectl exec -it user-service-xxxx -- cat /app/config/application.yml
```

📝 Sonraki Oturum Önizlemesi

Session 07: Kubernetes'te DevOps Pratikleri

- **CI/CD pipelines** with GitHub Actions
- **GitOps principles** ve FluxCD
- **Automated deployment** strategies
- **Container registry** integration
- **Complete automation** for e-commerce project

Hazırlık:

- GitHub account oluşturun
- CI/CD basics gözden geçirin

🙏 Teşekkürler!

Configuration Expert! ⚙️

Bugün öğrendiklerimiz:
✅ ConfigMaps ile configuration management
✅ Secrets ile güvenli data yönetimi
✅ Configuration injection patterns
✅ Security best practices
✅ E-commerce services'leri tamamen configure ettik

İletişim:

- GitHub: @huseyinbabal

Sonraki Oturum: Session 07 - Kubernetes'te DevOps Pratikleri